allows better control over argument number and type checking, and type conversions. Name of the parameter in function prototype has its scope limited to the prototype. This allows different parameter names in different declarations of the same function:

```
/* Here are two prototypes of the same function: */

int test(const char*)    /* declares function test */
int test(const char*p)   /* declares the same function
test */
```

Function prototypes greatly aid in documenting code. For example, the function Cf_Init takes two parameters: Control Port and Data Port. The question is, which is which? The function prototype

```
voidCf_Init(char *ctrlport, char *dataport);
```

makes it clear. If a header file contains function prototypes, you can that file to get the information you need for writing programs that call those functions. If you include an identifier in a prototype parameter, it is used only for any later error messages involving that parameter; it has no other effect.

Function Definition

Function definition consists of its declaration and a function body. The function body is technically a block – a sequence of local definitions and statements enclosed within braces {}. All variables declared within function body are local to the function, i.e. they have function scope.

The function itself can be defined only within the file scope. This means that function declarations cannot be nested.

To return the function result, use the return statement. Statement return in functions of void type cannot have a parameter – in fact, you can omit the return statement altogether if it is the last statement in the function body.

Here is a sample function definition:

```
/* function max returns greater one of its 2 arguments:
*/
```